



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Архітектура Комп'ютерів. Частина 1

Лабораторна робота №5

«Формування системи команд процесора, розробка програм і мікропрограм обробки
інформації в ЕОМ»

Виконав: студент 2 курсу ФІОТ, гр. ІО-41

Давидчук А. М.

Залікова книжка № 4106

Перевірів: *Верба О. А.*

Тема: «Формування системи команд процесора, розробка програм і мікропрограм обробки інформації в ЕОМ».

Мета: навчитися створювати системи команд процесорів, розробляти алгоритми, програми і мікропрограми обчислення функцій, використовувати двоадресні команди з регістровою адресацією та вводити дані з пристрою вводу-виводу.

Хід роботи

Підготовка до роботи

Номер варіанту – $4106_{10} = 0001\ 0000\ 0000\ 1010_2$.

$$a_7 = 0, \quad a_6 = 0, \quad a_5 = 0, \quad a_4 = 1, \quad a_3 = 0, \quad a_2 = 1, \quad a_1 = 0.$$

Для визначення варіанта лабораторної роботи використовуються біти a_7, a_6, a_5, a_3, a_1 . За комбінацією $a_5 a_3 a_1 = 000$ необхідно виконати операції з кодами 0, 1, 2 з таблиці методичних вказівок.

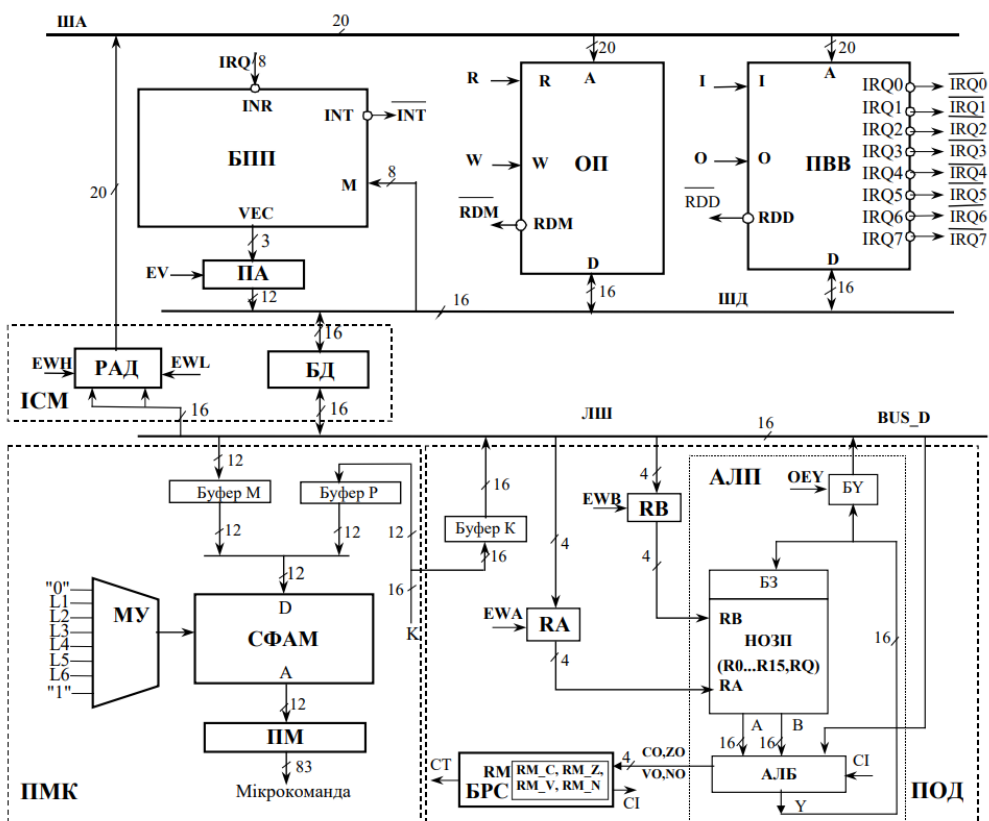
a_5	a_3	a_1	Операції
0	0	0	0, 1, 2
0	0	1	0, 2, 4
0	1	0	3, 4, 2
0	1	1	2, 3, 5

a_7	a_6	a_5	РС	РД
0	0	0	02H	04H
0	0	1	12H	14H
0	1	0	22H	24H
0	1	1	32H	34H

Код	Операція	Функція
0	$X + Y$	$F_1 = X + Y$
1	$X - Y$	$F_2 = X - Y$
2	$X \wedge Y$	$F_3 = X \wedge Y$
3	$X \vee Y$	–
4	$X \oplus Y$	–

Операнд X вводиться з пристрою введення через регістр даних ЗП, операнд Y розміщується в оперативній пам'яті. Результати F_1 , F_2 та F_3 записуються в ОП. Операнди подаються у доповняльному коді, розрядність операндів – 16 біт з двома знаковими розрядами.

Структура EOM



Формат команд та реєстрів стану і даних

Поле	Біти	Довжина	Призначення
F	15	1	формат команди: 0 – одноадресна, 1 – двоадресна
OP	14–11	4	код операції
TA	10	1	тип адресації: 0 – пряма, 1 – пряма реєстрова
A	9–0	10	адреса ОП, адреса порту або поля адрес реєстрів

Регістр	Призначення
РС	7-й біт містить ознаку готовності пристрою до обміну даними
РД	містить дані, що зчитуються із зовнішнього пристрою

Архітектура системи

Структура НОЗП

Регістр	Призначення
R1	операнд X
R2	операнд Y
R3	регістр команди
R4	лічильник команд
R5	стан зовнішнього пристрою
R6	результат $F_1 = X + Y$
R7	результат $F_2 = X - Y$
R8	результат $F_3 = X \wedge Y$

Структура ПМК

Адреса	Мікропрограма
0h	START
1h	TEST_ZP
2h	Z_JUMP
3h	INPUT_X
4h	GET_Y
5h	X_ADD_Y
6h	WRITE_F1
7h	X_SUB_Y
8h	WRITE_F2
9h	X_AND_Y
Ah	WRITE_F3
Bh	END

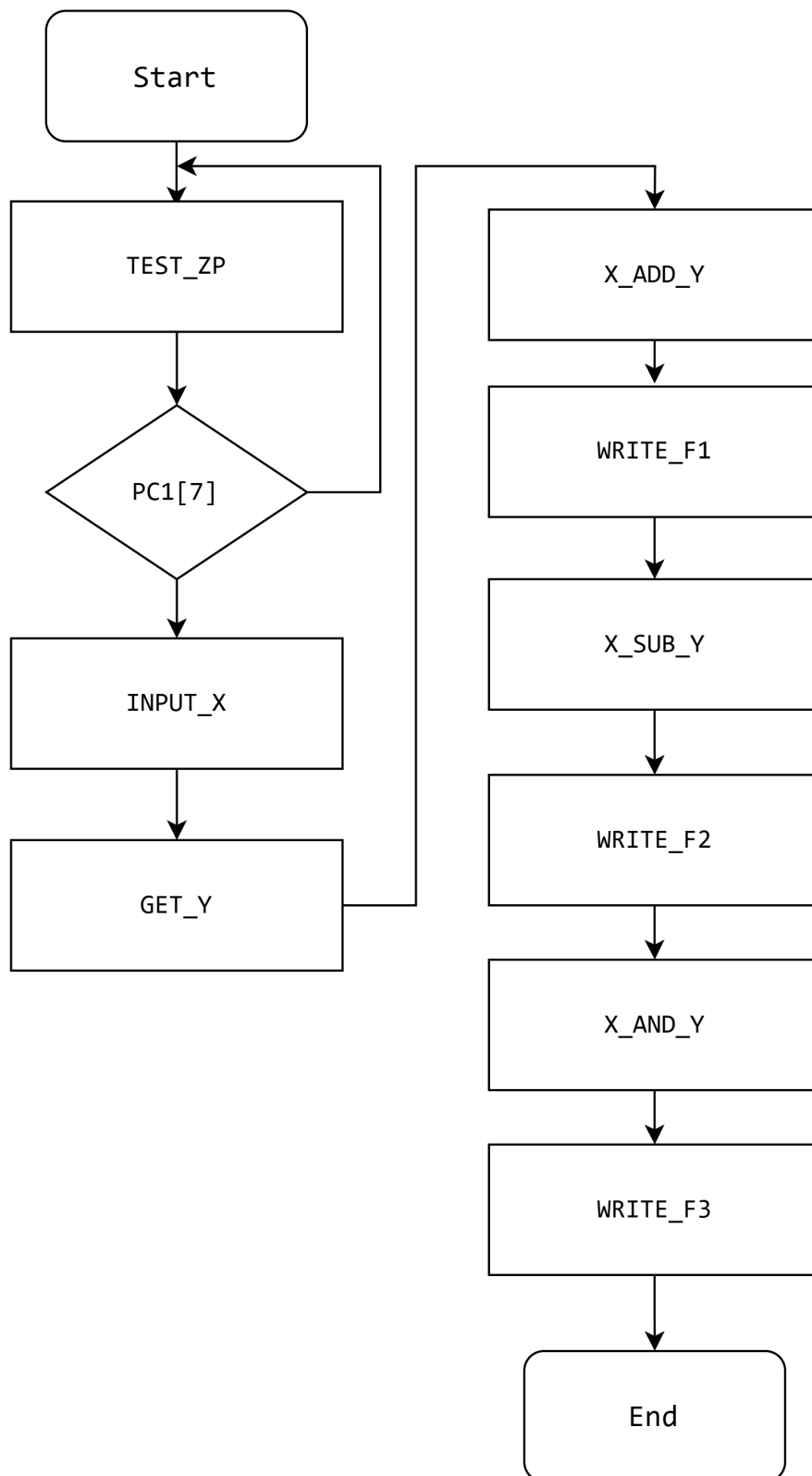
Структура ОП

Адреса	Вміст
0h	резерв
1h	значення Y
2h	результат F_1
3h	результат F_2
4h	результат F_3
10h–1Ah	програма

Структура ЗП

Адреса	Вміст
02h	РС, регістр стану
04h	РД, регістр даних

Змістовний алгоритм



Система команд

Команда	Код операції	Операція
TEST_ZP	1h	зчитування PC зовнішнього пристрою та перевірка готовності
Z_JUMP	2h	умовний перехід до адреси з поля команди
INPUT_X	3h	введення операнда X з РД зовнішнього пристрою
GET_Y	4h	зчитування операнда Y з оперативної пам'яті
X_ADD_Y	5h	двоадресна регістрова команда додавання $X + Y$
WRITE_F1	6h	запис результату F_1 в ОП
X_SUB_Y	7h	двоадресна регістрова команда віднімання $X - Y$
WRITE_F2	8h	запис результату F_2 в ОП
X_AND_Y	9h	двоадресна регістрова команда логічного І $X \wedge Y$
WRITE_F3	Ah	запис результату F_3 в ОП
END	Bh	завершення програми

Програма в оперативній пам'яті

Адреса	Код команди	Мнемоніка
10h	0802h	TEST_ZP 02h
11h	1010h	Z_JUMP 10h
12h	1804h	INPUT_X 04h
13h	2001h	GET_Y 01h
14h	AC62h	X_ADD_Y R2, R6
15h	3002h	WRITE_F1 02h
16h	BC72h	X_SUB_Y R2, R7
17h	4003h	WRITE_F2 03h
18h	CC82h	X_AND_Y R2, R8
19h	5004h	WRITE_F3 04h
1Ah	5800h	END

Обрахунки

Для першої перевірки обрано:

$$X = -2_{10} = FFFE_{16}, \quad Y = 52_{10} = 0034_{16}.$$

$$F_1 = X + Y = -2 + 52 = 50_{10} = 0032_{16},$$

$$F_2 = X - Y = -2 - 52 = -54_{10} = FFC A_{16},$$

$$F_3 = X \wedge Y = FFFE_{16} \wedge 0034_{16} = 0034_{16}.$$

Для другої перевірки обрано:

$$X = -10_{10} = FFF6_{16}, \quad Y = -4_{10} = FFFC_{16}.$$

$$F_1 = X + Y = -10 - 4 = -14_{10} = FFF2_{16},$$

$$F_2 = X - Y = -10 - (-4) = -6_{10} = FFFA_{16},$$

$$F_3 = X \wedge Y = FFF6_{16} \wedge FFFC_{16} = FFF4_{16}.$$

Мікропрограма

```
link L1: ct
link L2: rdm
link L3: rdd

link ewh: 10
link RB: 7, 6, 5, 4
link RA: 3, 2, 1, 0
link M: z, z, z, z, z, z, z, z, 14, 13, 12, 11

\ Variant: a7 a6 a5 a4 a3 a2 a1 = 0 0 0 1 0 1 0
\ a5 a3 a1 = 000 -> operatsii 0, 1, 2
\ F1 = X + Y
\ F2 = X - Y
\ F3 = X and Y
\ a7 a6 a5 = 000 -> RS = 02h, RD = 04h

accept dev[0]: I, 02h, 04h, 3, 3
accept dev_buf[0]: 0FFFEh \ X = -2 u DK

accept R1: 0000h \ X u DK
accept R2: 0000h \ Y u DK
accept R3: 0000h \ RK - reiestr komandy
accept R4: 0010h \ LK - lichylnyk komand
accept R5: 0000h \ stan ZP
accept R6: 0000h \ F1 = X + Y
accept R7: 0000h \ F2 = X - Y
accept R8: 0000h \ F3 = X and Y

\ Dani v OP
dw 0h: 0000h \ rezerv
dw 1h: 0034h \ Y = 52 u DK
dw 2h: 0000h \ F1
```

```

dw 3h: 0000h \ F2
dw 4h: 0000h \ F3

\ Programa v OP, pochatok z 10h
dw 10h: 0000100000000010% \ TEST_ZP, kod 1, adres RS = 02h
dw 11h: 0001000000010000% \ Z_JUMP, kod 2, adres 10h
dw 12h: 0001100000000100% \ INPUT_X, kod 3, adres RD = 04h
dw 13h: 0010000000000001% \ GET_Y, kod 4, adres OP 1h

dw 14h: 1010110001100010% \ X_ADD_Y, kod 5, RA=R2, RB=R6
dw 15h: 0011000000000010% \ WRITE_F1, kod 6, adres OP 2h

dw 16h: 1011110001110010% \ X_SUB_Y, kod 7, RA=R2, RB=R7
dw 17h: 0100000000000011% \ WRITE_F2, kod 8, adres OP 3h

dw 18h: 1100110010000010% \ X_AND_Y, kod 9, RA=R2, RB=R8
dw 19h: 0101000000000100% \ WRITE_F3, kod Ah, adres OP 4h

dw 1Ah: 0101100000000000% \ END, kod Bh

\ Vektory komand u PMK
org 0h {cjp nz, START;}

org 1h {cjp nz, TEST_ZP;}
org 2h {cjp nz, Z_JUMP;}
org 3h {cjp nz, INPUT_X;}
org 4h {cjp nz, GET_Y;}
org 5h {cjp nz, X_ADD_Y;}
org 6h {cjp nz, WRITE_F1;}
org 7h {cjp nz, X_SUB_Y;}
org 8h {cjp nz, WRITE_F2;}
org 9h {cjp nz, X_AND_Y;}
org 0Ah {cjp nz, WRITE_F3;}
org 0Bh {cjp nz, END;}

macro INC REG: {add REG, REG, z, nz;}
macro ADDRESS: {or nil, R3, z; oey; ewl;}
macro NEXT_COM: {cjp nz, NEXT;}

NEXT
{INC R4;} \ LK = LK + 1

START
{xor nil, R0, R0; oey; ewh;} \ starshi rozriady adresy = 0
{or nil, R4, z; oey; ewl;} \ adres potochnoi komandy z LK
{cjp rdm, cp; R; or R3, BUS_D, z;} \ chytannia komandy v RK
{or nil, R3, z; oey; jmap;} \ perekhid za kodom operatsii

TEST_ZP
{ADDRESS;} \ adres RS z adresnoho polia komandy
{cjp rdd, cp; I; or R5, BUS_D, z;} \ chytannia RS u R5
{and nil, R5, 0080h; load RM, flags;} \ perevirka bita hotovnosti
{NEXT_COM;}

```



```

Z_JUMP
{cjp not RM_Z, ZJ_NEXT;}          \ yakshcho ZP hotovyi, idemo dali
{and R4, R3, 03FFh;}              \ LK = adres z komandy
{cjp nz, START;}

ZJ_NEXT
{NEXT_COM;}

INPUT_X
{ADDRESS;}                        \ adres RD z adresnoho polia komandy
{cjp rdd, cp; I; or R1, BUS_D, z;} \ R1 = X
{NEXT_COM;}

GET_Y
{ADDRESS;}                        \ adres Y v OP
{cjp rdm, cp; R; or R2, BUS_D, z;} \ R2 = Y
{NEXT_COM;}

X_ADD_Y
{ADDRESS; ewa; ewb;}              \ RA=R2, RB=R6
{or RB, R1, z;}                   \ RB = X, tobto R6 = X
{add RB, RB, RA, z;}              \ RB = RB + RA, tobto R6 = X + Y
{NEXT_COM;}

WRITE_F1
{ADDRESS;}                        \ adres F1 v OP
{cjp rdm, cp; or nil, R6, z; oey; W;} \ zapis R6 v OP
{NEXT_COM;}

X_SUB_Y
{ADDRESS; ewa; ewb;}              \ RA=R2, RB=R7
{or RB, R1, z;}                   \ RB = X, tobto R7 = X
{sub RB, RB, RA, nz;}             \ RB = RB - RA, tobto R7 = X - Y
{NEXT_COM;}

WRITE_F2
{ADDRESS;}                        \ adres F2 v OP
{cjp rdm, cp; or nil, R7, z; oey; W;} \ zapis R7 v OP
{NEXT_COM;}

X_AND_Y
{ADDRESS; ewa; ewb;}              \ RA=R2, RB=R8
{or RB, R1, z;}                   \ RB = X, tobto R8 = X
{and RB, RB, RA;}                 \ RB = RB and RA, tobto R8 = X and Y
{NEXT_COM;}

WRITE_F3
{ADDRESS;}                        \ adres F3 v OP
{cjp rdm, cp; or nil, R8, z; oey; W;} \ zapis R8 v OP
{NEXT_COM;}

END

```

Результат виконання

Після виконання мікропрограми в оперативній пам'яті мають бути записані:

Адреса	Перша перевірка	Друга перевірка
M[2h]	0032h	FFF2h
M[3h]	FFCAh	FFFAh
M[4h]	0034h	FFF4h

Результат виконання для $X = FFFEh$ та $Y = 0034h$:

Результат виконання для $X = FFF6h$ та $Y = FFFCh$:

Як видно зі скріншотів, отримані значення в оперативній пам'яті збігаються з ручними обрахунками.

Висновок

У результаті виконання лабораторної роботи було сформовано систему команд процесора для обчислення трьох функцій $F_i = (X \circ Y)$, розроблено програму в машинних кодах і мікропрограму на мнемонічному мікроасемблері. Для мого варіанта реалізовано операції $X + Y$, $X - Y$ та $X \wedge Y$. Операнд X зчитується із зовнішнього пристрою після програмної перевірки біта готовності, операнд Y зчитується з оперативної пам'яті, а результати записуються назад в ОП. Працездатність програми перевірено для двох наборів вхідних даних; отримані результати збігаються з ручними розрахунками.

Відповіді на контрольні питання

1. Охарактеризуйте етап дешифрування (розпакування) команд.

На етапі дешифрування команда зчитується з оперативної пам'яті до регістра команд, після чого з її полів виділяються формат, код операції, тип адресації та адресне поле. Код операції використовується для переходу до відповідної мікропрограми в ПМК, а адресне поле – для формування адреси операнда або вибору регістрів при прямій регістровій адресації.

2. Як сформувати двоадресну команду при прямій регістровій адресації?

При прямій регістровій адресації адресне поле не використовується повністю для адреси ОП, тому його можна поділити на два підполя адрес регістрів. Наприклад, молодші біти поля команди подаються на RA, а старші – на RB. Так формується команда, яка задає одразу два регістрові операнди.

3. Як забезпечити зчитування і запис даних у пам'ять з урахуванням швидкодії пам'яті?

Після встановлення адреси на шині та формування сигналу читання або запису процесор повинен чекати сигналу готовності пам'яті RDM. Якщо пам'ять ще не готова, мікропрограма залишається в циклі очікування. Це узгоджує швидкодію процесора і пам'яті та запобігає зчитуванню або запису некоректних даних.

4. Як записати інформацію в RA і RB, навіщо використовуються зазначені регістри?

Інформація в RA і RB записується під час розпакування команди за допомогою керуючих сигналів `load RA` і `load RB` або через макрокоманду, що виконує розкодування адресних полів. RA і RB зберігають адреси регістрів НОЗП, які підключаються до входів АЛП, тому вони потрібні для виконання регістрових двоадресних команд.

5. Поясніть призначення директиви мікроасемблера MACRO.

Директива `MACRO` дозволяє описати власну мнемонічну мікрокоманду, яка розгортається в одну або кілька мікрооперацій. Її використовують для повторюваних дій, наприклад інкременту лічильника команд або формування адреси, щоб зробити мікропрограму коротшою і зрозумілішою.

6. Як забезпечити обмін даними між процесором і ЗП в програмному режимі?

У програмному режимі процесор спочатку звертається до регістра стану ЗП і перевіряє біт готовності. Якщо пристрій готовий, виконується читання з регістра даних або запис у нього за допомогою сигналів I/O і очікування сигналу RDD. Якщо пристрій не готовий, програма виконує умовний перехід до повторної перевірки.